

Architecture Summary

Architecture, data flow, and security model, for enterprise and technical architects

Product	Anthropic AI Assistant for Qlik Sense (extension + proxy)
Version	v0.5.5 · build 45
Author	mabaeyens
Date	September 15, 2026
Language	English

Notice — independent project, not a Qlik product

This extension is an independent, author-maintained project. It is not a Qlik product, a Qlik offering, or a Qlik-supported integration. Support is provided through GitHub Issues, not through Qlik's own support channels. Neither Qlik nor the author accept any liability for any issue, data exposure, cost, or damage arising from its use in any customer or production environment.

Asking a question sends the selected chart's data out of your environment to an LLM (Anthropic, via the proxy), unless you pick a local model, in which case the data stays on your own infrastructure. Do not use it with sensitive, regulated, or personal data without explicit authorization.

1. Executive summary

The Anthropic AI Assistant (v0.5.5, build 45) is a Qlik Sense visualization extension that analyzes the selected chart's data using Anthropic's Claude model family, or a local model via Ollama. As of v0.5.0 the system is a hardened pair: the extension (client) and a Node.js proxy (server, currently v2.0.2). It is an independent project, not a Qlik product.

- Target platform: client-managed Qlik Sense on Windows (Desktop and Enterprise/QSEoW), version 3.0 or later. Qlik Cloud is out of scope (Cloud already ships native AI assistants).
- Integration pattern: the browser NEVER calls api.anthropic.com directly. Every request goes through the proxy, which holds the key server-side and validates the caller's Qlik session before any upstream call.
- No client-side build step: vanilla JavaScript (AMD) loaded through Qlik's RequireJS system.
- The proxy is an independent Node.js runtime, versioned separately (proxy-vX.Y.Z), deployable as an auto-start Windows service.

2. Module architecture (extension)

All modules follow the RequireJS define([deps], factory) pattern. The entry point, AnthropicExtension.js, delegates to js/main.js, which exposes paint(element, layout), called by Qlik on every render cycle. UI/selection initialization happens once per instance (a first-run guard); the properties (model, proxy URL, local URL, log level) are applied on every call to paint().

Bundled libraries (no CDN): marked (Markdown to HTML) and DOMPurify (fail-closed HTML sanitization). There is no client-side crypto library: with no key in the browser, there is nothing left to encrypt.

Module	Responsibility
AnthropicExtension.js	Entry point (RequireJS shim into js/main).
js/main.js	Qlik's paint() cycle, properties panel, first-run guard, boot config validation.
js/ui-controller.js	Floating panel, conversation thread, chart selection, request assembly, single-flight request lifecycle (busy/abort), payload warnings.

Module	Responsibility
js/data-collector.js	Extracts the selected object's full hypercube and collects the app's data-model context (tables/fields + master items) once per session, memoizing the in-flight promise, not the resolved value.
js/data-format.js	Compacts and formats the data for the LLM; estimates tokens and trims rows to stay under budget.
js/anthropic-api.js	Builds the message, resolves the single proxy transport, and POSTs with credentials:include to forward the Qlik session. Supports streaming (SSE) with an automatic buffered fallback.
js/chart-builder.js	Parses Claude's chart spec, renders a live preview, and adds the chart to the sheet via the Qlik visualization API.
js/formatting.js	Renders the Markdown answer to sanitized HTML (DOMPurify, fail-closed).
js/config-validate.js	Validates configuration at boot; problems surface in the panel instead of breaking the render.
js/log.js	Level-gated console logging (ERROR/WARN/INFO/DEBUG) controlled by config.LOG_LEVEL.
js/template.js	The panel's inlined HTML markup (injected by ui-controller).
js/config.js	The single source of truth: model, proxy endpoints, tokens, limits, log level, and system prompt.

3. End-to-end request flow

- Select: the click is detected via the `qv-object-<id>` DOM class; the object's full hypercube is extracted (paged up to the allowed cell ceiling).
- App context (first request only): `getAppContextCached()` fetches real table and field names plus master items (dimensions/measures with their expressions) and caches them for the session.
- Format and budget: the data is compacted and tokens estimated, trimming rows per `MAX_ROWS`.
- Assemble: `ui-controller` builds `{ userPrompt, chartData, context, systemPrompt, history }`. A payload over ~65 KB prompts for confirmation; over 1 MB it is blocked client-side.
- Context guard: if the estimate exceeds the model's window, the user can truncate the data or cancel.
- Transport: a single route to the proxy (`config.API.PROXY_URL` for hosted models; `config.API.LOCAL_URL` for local models). The browser carries no key at all.
- Call: POST to the proxy with `credentials:include`, so the browser automatically attaches the Qlik session cookie. The proxy validates the session, injects the key, and forwards upstream.
- Render: the answer renders as sanitized Markdown in the thread, with a copy button; with streaming on, token by token.
- Suggested chart (optional, no extra LLM call to create it): `chart-builder` previews it and, in edit mode, adds the object to the sheet as the logged-in user.

4. The hardened proxy

The proxy (`proxy/` folder, Node.js + Express) is where the key lives and where the caller is authenticated. Its hardening layers (specs P01-P07):

Layer	Function
Credential custody (P01)	The Anthropic key is held server-side (<code>ANTHROPIC_API_KEY</code> environment variable), injected per request; any client-supplied key header is stripped.

Layer	Function
Caller authentication (P02)	Validates the Qlik session against the QPS API over mutual TLS before any upstream call. In a same-site deployment the browser sends the session cookie automatically.
Concurrency & resilience (P03)	Admission control with global and per-user in-flight ceilings, a bounded FIFO queue (503 + Retry-After on overflow), streaming backpressure, and an orderly drain on shutdown.
Validation & allowlist (P04)	Per-route schema validation, a body-size cap, and a server-side model allowlist: invalid, oversized, or disallowed requests are rejected.
Transport & CORS (P05)	A strict origin allowlist, security headers, a modern TLS floor, and a per-IP rate limit.
Observability & service (P06)	Structured JSON logs with level gating, a separate audit log (no bodies or secrets), /health, /ready, /metrics, boot-time config validation, and an auto-start Windows service wrapper.

5. API specification (proxy)

Routes

POST /api/anthropic -> forwards to api.anthropic.com/v1/messages (key injected by the proxy)
 POST /api/ollama -> forwards to a local Ollama server (OpenAI chat-completions shape)
 GET /health -> liveness GET /ready -> readiness GET /metrics -> counters

Client request to the proxy

POST <config.API.PROXY_URL>
 Content-Type: application/json
 Cookie: X-Qlik-Session=<session> (sent automatically by the browser, same-site)
 (the browser sends NO API key; the proxy adds its own server-side)

Default system prompt: “You are a business analyst and expert Qlik Sense user. Be concise. Always aggregate the data and show absolute values and percentages. Focus on insights that would help business decision making. Present your analysis in a structured format with bullet points for key findings.”

Parameter	Value
Proxy endpoint (hosted)	https://<qlik-host>:3000/api/anthropic
Proxy endpoint (local)	https://<qlik-host>:3000/api/ollama
Default model	claude-haiku-4-5
Available models	claude-haiku-4-5, claude-sonnet-5, claude-opus-5, Ministral 3 8B/3B (local)
max_tokens	4000
Timeout	60,000 ms (hosted) / 300,000 ms (local model)
Context window (approx.)	200,000 tokens (hosted) / 8,192 (local Ministral)

6. Security model

Where the API key lives

- On the proxy, server-side (ANTHROPIC_API_KEY environment variable). The browser never sees or stores it.
- The proxy strips any key header a client might send and injects its own per request.
- There is no browser-side encryption, localStorage, or bundled passphrase: that risk surface, present in versions before v0.5.0, has been removed entirely.

Authentication

- The proxy independently validates the caller's Qlik session (it never trusts an identity asserted by the client) and resolves the real user for per-user limits and audit.
- Default carrier: the same-site session cookie (X-Qlik-Session). A prefixed virtual proxy uses X-Qlik-Session-<prefix>.

Data leaving the browser

- On every request: the selected chart's hypercube contents, the user's question, and the history (up to 12 messages).
- On the first request only: real table and field names, plus master items with their expressions.
- Never sent: Qlik credentials to the LLM, or data from other sheets/apps. All traffic is HTTPS.
- With a local model (Ollama), data flows from the browser to the proxy and from there to Ollama on your own network, it never reaches the public internet.

7. Deployment model

- Extension (no build): JavaScript + RequireJS; no npm, transpiling, or bundling at runtime. Changes take effect on browser reload.
- Proxy: a Node.js runtime on the Qlik node. Automated setup with proxy/scripts/setup.ps1 (runtime dependencies via npm ci --omit=dev, a TLS certificate, .env, and optionally an auto-start Windows service).
- Recommended proxy placement: same-site with the hub, so the browser sends the Qlik session cookie automatically and the proxy validates it against QPS over mutual TLS.
- Proxy certificate: in production, CA-signed for the proxy's hostname, so the Qlik page trusts it without any manual import.

8. Configurable limits

Parameter	Value	Purpose
MAX_ROWS	1000	Maximum rows sent to the LLM per chart.
MAX_FETCH_CELLS	50000	Ceiling on cells read from a hypercube; beyond it, data is truncated with a notice.
MAX_CELLS_PER_PAGE	10000	Cells per getHyperCubeData page request.
LARGE_TABLE_CELLS	25000	Threshold above which the user is warned of a large/truncated table.
FETCH_PAGE_CONCURRENCY	4	Max concurrent page requests to the engine.
MAX_FIELDS	500	Cap on the field list sent as context.
WARN / MAX_PAYLOAD_BYTES	~65 KB / 1 MB	Confirmation warning / hard block on payload size.
HISTORY_MAX	12	Prior messages sent per request.
MAX_GLOBAL_INFLIGHT (proxy)	24	Max simultaneous upstream calls, across all users.
MAX_USER_INFLIGHT (proxy)	3	Max concurrent calls per authenticated user.

Map visualizations are not supported.

9. Data governance and network considerations

- Every analysis with a hosted model means chart data leaves toward an external LLM (Anthropic); the residency and retention of that data are outside the customer's direct control.
- The proxy centralizes control: it is the only point that reaches the internet, and it enforces the model allowlist, rate limits, audit logging, and Qlik session validation.
- For data that must not leave the perimeter, use a local model (Ollama) through the proxy.